A continuación, presento el Modelo Entidad-Relación (MER) optimizado y el diseño de las transacciones ACID.

1. Esquema de Base de Datos (Tablas Principales)

El diseño minimiza la redundancia y optimiza las consultas más frecuentes: la carga del garaje (álbum) y la validación del inventario para los intercambios.

Entidades Core

users: Almacena el perfil y la billetera.

id (PK, UUID)

username (VARCHAR)

email (VARCHAR, UNIQUE)

racing_credits (INT, DEFAULT 0)

created_at (TIMESTAMP)

vehicles_catalog: El diccionario maestro de todas las figuritas del juego.

id (PK, INT)

name (VARCHAR) - Ej: "Fiat Uno 1.4s Confort 1997"

category (VARCHAR) - Ej: "Clásicos Urbanos"

rarity_tier (VARCHAR) - Ej: "Común", "Raro", "Legendario"

image_url (VARCHAR)

Entidades de Inventario

user_inventory: Qué tiene cada piloto. Se utiliza un modelo agregado para ahorrar millones de filas.

user_id (PF/FK -> users.id)

vehicle_id (PK/FK -> vehicles_catalog.id)

is_pasted (BOOLEAN, DEFAULT FALSE) - True si ya está en el álbum.

unpasted_count (INT, DEFAULT 0) - Cantidad disponible en el inventario (sirve para pegar o intercambiar).

(PK Compuesta: user_id + vehicle_id)

packs_inventory: Sobres sin abrir.

user_id (PK/FK -> users.id)

pack_type (VARCHAR) - Para el MVP, "Standard Crate".

quantity (INT, DEFAULT 0)

Entidades de Economía e Intercambio

trailer_sessions: Registro temporal para evitar trampas al ganar créditos.

session_id (PK, UUID)

user_id (FK -> users.id)

trailer_id (VARCHAR)

started_at (TIMESTAMP)

status (VARCHAR) - "PENDING", "COMPLETED", "FRAUD"

trades: El mercado global de intercambios.

trade_id (PK, UUID)

offer_user_id (FK -> users.id)

offer_vehicle_id (FK -> vehicles_catalog.id)

request_vehicle_id (FK -> vehicles_catalog.id)

status (VARCHAR) - "OPEN", "MATCHED", "CANCELLED"

created_at (TIMESTAMP)

2. Manejo de Créditos por Tráilers (Anti-Cheat)

No podemos confiar en que el cliente (frontend) nos envíe un simple POST /api/add-credits. El flujo debe ser controlado por el servidor usando la tabla trailer_sessions:

1. Inicio: El usuario hace clic en "Ver Tráiler". El frontend solicita un token al backend.

2. Registro: El servidor inserta un registro en trailer_sessions con el started_at actual (hora del servidor) y estado PENDING. Devuelve el session_id.

3. Visualización: El usuario ve el video de 30 segundos.

4. Reclamación: Al terminar, el frontend envía POST /api/claim-trailer-reward con el session_id.

5. Validación (Regla de Negocio): El servidor busca la sesión. Calcula la diferencia entre NOW() y started_at.

Si la diferencia es menor a la duración del video (ej. 28 segundos), el usuario usó un speedhack o saltó el video. Se marca como FRAUD y no se otorgan créditos.

Si es correcta y no ha excedido el límite diario, se actualiza el estado a COMPLETED y se suman los créditos al usuario.

3. Transacciones ACID (Evitando la pérdida o clonación de figuritas)

El mayor riesgo en estos sistemas es la concurrencia: dos procesos intentando modificar el inventario al mismo tiempo (ej. el sistema de matchmaking ejecuta un intercambio justo cuando el usuario intenta pegar esa misma figurita).

Para solucionarlo, utilizaremos Bloqueos Pesimistas (Pessimistic Locking) mediante la cláusula FOR UPDATE en SQL dentro de una transacción.

Escenario: Ejecución de un Intercambio (Matchmaking Exitoso)

```
BEGIN TRANSACTION;

-- 1. Bloquear los registros de inventario de AMBOS usuarios involucrados para que no puedan ser alterados por otro proceso.
SELECT unpasted_count FROM user_inventory
WHERE user_id = 'UserA' AND vehicle_id = 'AutoOfrecido' FOR UPDATE;

SELECT unpasted_count FROM user_inventory
WHERE user_id = 'UserB' AND vehicle_id = 'AutoPedido' FOR UPDATE;

-- (Lógica de servidor en Node/Python verifica que unpasted_count > 0 en ambos casos)

-- 2. Descontar las figuritas
UPDATE user_inventory SET unpasted_count = unpasted_count - 1
WHERE user_id = 'UserA' AND vehicle_id = 'AutoOfrecido';
```

```
UPDATE user_inventory SET unpasted_count = unpasted_count - 1

WHERE user_id = 'UserB' AND vehicle_id = 'AutoPedido';


-- 3. Sumar las figuritas recibidas (utilizando UPSERT/ON CONFLICT por si es la primera vez que la obtienen)

INSERT INTO user_inventory (user_id, vehicle_id, unpasted_count)

VALUES ('UserA', 'AutoPedido', 1)

ON CONFLICT (user_id, vehicle_id) DO UPDATE SET unpasted_count = user_inventory.unpasted_count + 1;


INSERT INTO user_inventory (user_id, vehicle_id, unpasted_count)

VALUES ('UserB', 'AutoOfrecido', 1)

ON CONFLICT (user_id, vehicle_id) DO UPDATE SET unpasted_count = user_inventory.unpasted_count + 1;


-- 4. Cerrar la oferta en el mercado

UPDATE trades SET status = 'MATCHED' WHERE trade_id = 'TradeUUID';


COMMIT;
```